

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..44

SCALABILITY

 Copy as Markdown

Scalability

When traffic jumps, the first bottlenecks are usually a blocked Dart event loop, an undersized database pool across many nodes, or shared state that only lives in one process. This page covers how Serverpod scales out request-serving instances, and what to change so CPU work, Postgres, auth, caching, and long-lived connections keep up.

Elastic scale-out of request handlers is already covered by [server roles](#). The sections below cover what still bites at high daily active users. Built-in read replicas and a Serverpod-owned Postgres tuning recipe are not available today; use cloud provider guides and the escape hatches noted here.

How Serverpod scales

Each Serverpod process runs on one Dart isolate and one event loop. CPU-heavy work on that isolate delays request handling for every concurrent call on the same process.

HTTPS APIs with JWT auth are largely stateless and scale behind a load balancer. Features that keep in-process state (local caches, local server events, long-lived streams without a shared bus) need either sticky routing, Redis, or a redesign so state lives outside the process.

For serverless or auto-scaled fleets, run request nodes in the `serverless` role (or `monolith` if you accept in-process maintenance), and run `maintenance` on a schedule for future calls and health metric collection. See [Hosting elsewhere](#) and [Scheduling overview](#).

Split request and maintenance work

Use roles so request capacity and background work do not share the same scaling curve:



Role	Serves requests	Runs future calls and health metrics
<code>monolith</code>	Yes	Yes (continuous)
<code>serverless</code>	Yes	No
<code>maintenance</code>	No	Once, then exit

```
dart run bin/main.dart --mode production --role serverless
```

Schedule a separate process in the `maintenance` role (for example once per minute) when request nodes run as `serverless`. On pure request nodes you can also set `futureCallExecutionEnabled` to `false`; see [Future call configuration](#).

Offload CPU work to isolates

Image decoding, password hashing, and other CPU-bound work should not run on the request isolate. Use Dart's `Isolate.run` with plain data only:

```
Future<List<int>> buildThumbnail(List<int> bytes) async {
  return Isolate.run(() {
    // CPU-heavy decode, resize, and encode.
    // Pass only serializable data in and out.
    return processImage(bytes);
  });
}
```

Do not pass a `Session`, database connection, or Redis client into the isolate. Do I/O on the main isolate after the isolate returns.

There is no first-party Serverpod helper for this pattern. For sustained background workloads, raise `futureCall.concurrencyLimit` carefully (default `1`), or plan for a dedicated worker model as your product grows.

Optimize Postgres queries

Declare indexes in your models for columns you filter and sort on. Without an index that covers an `ORDER BY`, Postgres may sort the full result set before applying `OFFSET` / `LIMIT`. See [Indexing](#).

Prefer [cursor-based pagination](#) over deep `offset` pages on large tables: filter with `where: (t) => t.id > lastId`, `orderBy` on `id`, and a `limit`.

At high scale, treat wide `OR` / `inSet` filters carefully, especially expressions like `(A OR B) AND (C OR D)`. Those shapes can explode into many indexed paths. Prefer a denormalized flag, a `UNION` of simpler indexed queries, or a narrower filter design when those queries dominate load.

Size the connection pool

Each Serverpod process opens its own pool. The default `database.maxConnectionCount` is `10` (override with `SERVERPOD_DATABASE_MAX_CONNECTION_COUNT`). Across a fleet:

$$(\text{number of nodes} \times \text{maxConnectionCount}) + \text{headroom} \leq \text{Postgres max_connections}$$

Leave headroom for Insights, maintenance jobs, migrations, and admin tools. Unlimited pools (a negative value, or omitting the key) are easy to misconfigure under auto-scaling. See [Database connection](#).

Scale Postgres itself

Assigning CPU, memory, and storage to Postgres is owned by your cloud provider (RDS instance class, Cloud SQL machine type, and similar). Point your operators at those guides rather than copying example parameters into Serverpod config.

Serverpod does not ship a read-only query API (there is no `MyTable.dbro`) and does not auto-route reads to replicas. Practical options today:

- Put a proxy or cloud read endpoint in front of the database (PgBouncer, RDS Proxy, or your provider's reader endpoint) when the proxy can classify traffic.
- Or wrap `session.db` with a [database interceptor](#) that routes selected reads yourself.

Treat third-party Postgres tuning examples as illustrative for your workload and instance size, not as Serverpod defaults.

Prefer JWT for auth at scale

Modern projects use the [JWT token manager](#). Access-token verification is local and does not query the database on every request. Refresh still hits the database; that path is far less frequent than ordinary API calls.

[Server-side sessions](#) are the opposite tradeoff: a database lookup on validation, with immediate revocation. Prefer JWT when request volume is the constraint. Legacy auth key handlers that load a row per request do not belong on a high-QPS fleet.

Share state across instances with Redis

Local caches (`session.caches.local` and `localPrio`) stay on one process. For values every instance must see, use the [global cache](#) with Redis enabled.

Cross-instance `server events` need Redis as well. `MessageScope.global` throws a `StateError` without Redis, and the default `MessageScope.auto` falls back to local delivery, so without Redis events silently stop reaching other instances.

Enable [Redis in production config](#) when you run more than one request node that shares cache entries or broadcast events.

Choose streams carefully

[Streaming endpoints](#) keep a WebSocket session open for the life of the stream. That is the right model for chat, multiplayer, and live dashboards. At very high concurrent clients per node, long-lived connections consume file descriptors and complicate load balancing and autoscaling.

When fan-out volume is the problem, prefer short HTTPS RPCs for client-to-server work and an external push service (FCM, APNs, or similar) for server-to-client wakeups. Firebase in Serverpod is an auth identity provider, not a push delivery product; wire push as an integration outside the framework.

Keep `websocketPingInterval` (default 30 seconds) in mind under high connection counts: each open socket pays keepalive cost.

Configuration cheat sheet

Setting	Default	Why it matters at scale
<code>role / SERVERPOD_SERVER_ROLE</code>	<code>monolith</code>	Split request nodes from maintenance work.
<code>database.maxConnectionCount</code>	<code>10</code>	Pool size times node count must fit Postgres.
<code>redis.enabled</code>	<code>true</code> when a <code>redis</code> section is present	Required for shared cache and global events.
<code>maxRequestSize</code>	<code>524288</code>	Large uploads increase memory pressure.
<code>websocketPingInterval</code>	<code>30</code> (seconds)	Keepalive cost under many open streams.
<code>futureCall.concurrencyLimit</code>	<code>1</code>	Caps background CPU and database load.

Setting	Default	Why it matters at scale
<code>sessionLogs.persistEnabled</code>	on when a database is configured	Extra database writes per request when on.
<code>healthCheckInterval</code>	1 minute	Metrics write load; aggressive liveness elsewhere can cascade restarts.

Full keys and environment variables: [Configuration reference](#).

Limits

- No first-class read-replica routing or `dbro` API; use a proxy or a database interceptor.
- No Serverpod-owned production Postgres tuning profile; follow your cloud database guide.
- Isolate offloading is a Dart convention in your code, not a framework API.
- Terraform and sample AWS layouts in community examples are starting points, not production scale blueprints.

Related

- [Server roles](#): which processes serve traffic versus maintenance.
- [Hosting elsewhere](#): roles and Docker on your own host.
- [Indexing](#): declare indexes for filters and sorts.
- [Caching](#): local versus Redis-backed global cache.
- [Redis](#): enabling Redis, the password, and connecting a GUI.
- [Health checks](#): probes and metric collection under load.
- [Configuration reference](#): every scale-related env var.